

D3: Dynamic Programming

When the interview is going really well and the interviewer asks you a dynamic programming problem



Sec 16: Fr 12:30pm-1:30pm FXB 1024
IA: Nick Frieler

Agenda

- ▶ Dynamic programming
 - ▶ Dynamic
 - ▶ Dyn
 - ▶ amic
 - ▶ programming
 - ▶ program
 - ▶ pro
 - ▶ gram
 - ▶ ming
 - ▶ m
 - ▶ ing

When can we use DP?

DP problems need 2 key features

- ▶ The “principle of optimality” or “optimal substructure”
 - ▶ Optimal solutions to a large instance can be obtained from multiple optimal solutions to smaller instances
 - ▶ There may be more than one optimal solution, but we only need at least one can be obtained like this
 - ▶ This should remind you of recursion
- ▶ Overlapping subproblems
 - ▶ Depending on the problem, we may have to solve exponentially many subproblems
 - ▶ For example, a naive recursive implementation of $\text{Fibonacci}(n)$ requires solving $\sim 1.618^n$ subproblems
 - ▶ To make use of DP, these subproblems need to “overlap” so that we can reuse solutions we have already computed

The DP “cookbook”

Focus on the “value” version of the problem

- ▶ Instead of finding an optimal solution, focus on finding the value of an optimal solution
- ▶ For example:
 - ▶ Suppose you are in a motorcycle race and can only drive to the right or down
 - ▶ Each square you visit adds that number of minutes to your total time
 - ▶ You want to find the optimal path so you can win!
 - ▶ First, figure out how long your winning (optimal) race time will be

	4	7	6	6	1
2	6	6	8	6	6
6	3	4	6	2	0
6	5	2	0	5	7
4	5	6	6	9	3
6	3	9	0	6	

Make and use a recurrence

- Now that we are focusing on the value version of the problem, come up with a recurrence for that value and use it to get the optimal value
- For example:
 - Figure out the minimum time it takes to get to each square in the race grid via some sort of recurrence
 - The minimum total time to get to a given square should look something like:
 - Total time = Square time + min(left time, up time)

	4	7	6	6	1
2	6	6	8	6	6
6	3	4	6	2	0
6	5	2	0	5	7
4	5	6	6	9	3
6	3	9	0	6	

	4	11	17	23	24
2	8	14	22	28	30
8	11	15	21	23	23
14	16	17	17	22	29
18	21	23	23	32	32
24	24	32	23	29	

Get the optimal solution using backtracking

	4	7	6	6	1
2	6	6	8	6	6
6	3	4	6	2	0
6	5	2	0	5	7
4	5	6	6	9	3
6	3	9	0	6	

- ▶ Use the filled-in values to figure out the optimal solution
- ▶ For example:
 - ▶ Total time = Square time + min(left time, up time)
 - ▶ Based on our recurrence, we can see we should pick the previous square that has the smaller value
- ▶ Note that there's more than 1 optimal path
 - ▶ In the 3rd row, 2nd column, we could have backtracked either left or up from the 11 to one of the two 8s

	4	11	17	23	24
2	8	14	22	28	30
8	11	15	21	23	23
14	16	17	17	22	29
18	21	23	23	32	32
24	24	32	23	29	

Runtime analysis, YAY

- Analyze your algorithm to determine runtime
- Likely used a DP table
 - Bound the number of table entries filled
 - Bound the time to fill each entry
- For example:
 - Total time = Square time + min(left time, up time)
 - Based on our recurrence, we can see each table entry takes $\Theta(1)$ time to fill
 - If our grid is $n \times m$, there are $\Theta(n * m)$ entries
 - Therefore, runtime is $\Theta(n * m)$

	4	7	6	6	1
2	6	6	8	6	6
6	3	4	6	2	0
6	5	2	0	5	7
4	5	6	6	9	3
6	3	9	0	6	

	4	11	17	23	24
2	8	14	22	28	30
8	11	15	21	23	23
14	16	17	17	22	29
18	21	23	23	32	32
24	24	32	23	29	

Any questions before
discussion problems?

Discussion Problems

Q1: Who climbs 3 steps at once??

We define the function $CS(n)$ as follows:

There are n steps to climb. Return the number of ways there are for a person standing at the bottom of the stairs to reach the top if they can climb either 1, 2, or 3 steps at a time.

Devise a recurrence for computing $CS(n)$.

Q1: Who climbs 3 steps at once??

We define the function $CS(n)$ as follows:

There are n steps to climb. Return the number of ways there are for a person standing at the bottom of the stairs to reach the top if they can climb either 1, 2, or 3 steps at a time.

Devise a recurrence for computing $CS(n)$.

Recurrence:

For $n \geq 3$,

$$CS(n) = CS(n-1) + CS(n-2) + CS(n-3)$$

Base Cases:

$$CS(0) = CS(1) = 1$$

$$CS(2) = 2$$

Q2: McNuggets

You and some friends are really hungry after your Thirsty Thursday Theatrics, so you decide to online order some chicken McNuggets. There are several order sizes you can get, so you put each of the sizes into an array $M[1, \dots, n]$. Being the brains of this operation, your friends task you with figuring out if there's some combination of meals you can order so that your total is exactly t nuggets.

Devise an algorithm to solve this problem.

Q2: McNuggets

- ▶ Setup:
 - ▶ Define $S(i)$ as true or false depending on if you can purchase i nuggets
- ▶ Base Cases:
 - ▶ $S(0) = \text{true}$
 - ▶ $S(i) = \text{false}$ for all $i < 0$
- ▶ Recurrence:
 - ▶ If $S(i - M[1])$ is true, then $S(i)$ is true
 - ▶ If $S(i - M[2])$ is true, then $S(i)$ is true
 - ▶ ...
 - ▶ $S(i) = \text{OR}(S(i - M[j]))$ for $j = 1, \dots, n$
- ▶ Answer:
 - ▶ $S(t)$ is the truth value of whether or not we can buy exactly t nuggets

Q2: McNuggets

Now that you have successfully developed an algorithm to determine whether or not you can buy the nuggets, you thought you were in the clear. BUT, your friends are *really* hungry and keep asking you how long until the nuggets arrive.

Do a runtime analysis on an implementation of the recurrence that uses DP.

Q2: McNuggets

Now that you have successfully developed an algorithm to determine whether or not you can buy the nuggets, you thought you were in the clear. BUT, your friends are *really* hungry and keep asking you how long until the nuggets arrive.

Do a runtime analysis on an implementation of the recurrence that uses DP.

- Making a table for the values $S(0)$, $S(1)$, ..., $S(t)$, we have $t + 1 = \Theta(t)$ entries to fill out
- For each entry, we do an OR of n values (the number of meal sizes), so each entry takes $O(n)$ time to compute
- In total, the time to compute $S(t)$ is $O(t * n)$

Attendance



Q3: Longest Arithmetic Subsequence

$\text{LAS}(A, d)$: Given an array of integers A and a difference d , return the length of any longest arithmetic subsequence in A with difference d . That is, return the length of a longest (not necessarily contiguous) subsequence S of A such that $S[i+1] - S[i] = d$ for each i .

Give a recurrence relation for LAS suitable for DP.

Q3: Longest Arithmetic Subsequence

LAS(A, d): Given an array of integers A and a difference d , return the length of any longest arithmetic subsequence in A with difference d . That is, return the length of a longest (not necessarily contiguous) subsequence S of A such that $S[i+1] - S[i] = d$ for each i .

Give a recurrence relation for LAS suitable for DP.

- ▶ Let $L[i]$ for $i = 1, \dots, n$ be the longest arithmetic subsequence of $A[1, \dots, n]$ with difference d **ending at index i**
- ▶ Look at all of the values $L[j]$ where $j < i$ AND $A[i] - A[j] = d$
 - ▶ Add **+1** to the **biggest value**
 - ▶ If there are **no such j** , then set $L[i] = 1$
- ▶ Take the **max of the $L[i]$**